

NEED IT

Mobile App Design Brief

What the product does and what every screen needs.
How it looks is yours.

DOCUMENT	Design Brief — Mobile App (V1)
AUDIENCE	Frontend / UI Design
PLATFORM	Android & iOS, single codebase
VERSION	1.0 — October 2026
STATUS	For design

01 Purpose

This document tells you what the app must do and what each screen must contain. It deliberately says nothing about servers, APIs, databases, or authentication mechanisms — those are handled elsewhere. Your job is the experience: how it looks, how it flows, how it feels in a student's hand at 11pm when they need Maggi from outside campus.

Need It is a campus errand app. A student posts a request for items they need from shops outside campus. Another verified student — a helper — who is going out anyway accepts the request, buys the items, and hands them over at a campus pickup point. The requester pays the item cost plus a small helper fee. Money is held safely and released to the helper only after both sides confirm the handover.

02 The Product in One Minute

- **Two roles, one app.** The same person can be a customer today and a helper tomorrow. They pick a role at signup and can act in either.
- **Trust is the product.** Everyone is verified with a college email. Phone numbers stay hidden. Money only moves when both sides confirm.
- **The core loop:** post → accept → agree price → pay → buy → hand over → done.
- **Everything runs on order status.** What a user can do on a screen depends entirely on what state the order is in. The app always shows the current status clearly.

03 Users & Roles

ROLE	WHAT THEY DO	KEY SCREENS
Customer	Posts requests, approves prices, pays, receives items, rates the helper.	My Requests, Create Request, Request Detail, Pay, Handover
Helper	Browses open requests, accepts them, shops, delivers, earns the fee.	Feed, My Orders, Order Detail, Handover, Availability

Design implication: navigation should make switching between "my requests" and "helping" effortless — they are two halves of the same person, not two apps.

04 Order Lifecycle

Every order moves through a fixed set of statuses. These names and their order are part of the product's shared language — use them exactly as written in the UI.



- **Open** — posted, waiting for a helper.
- **Accepted** — a helper has claimed it. Chat opens. Price gets agreed.
- **Purchased** — payment confirmed, helper has bought the items.
- **Handed Over** — both sides confirmed the handover with a one-time code.
- **Completed** — closed. Optional rating.
- **Cancelled** — an end state reached from earlier stages. Show it distinctly, not as a failure color that implies error.

Fixed by product: status names, their order, and the fact that available actions depend on current status. **Yours:** how status looks — chips, timeline, color language, progress indicator.

05 Design Principles

- **Trust first.** The app handles students' money and their safety. It should feel calm, transparent, and professional — never playful about money.
- **One primary action per screen.** At any moment there is usually exactly one thing the user should do next. Make it unmistakable.
- **Show the money split.** Item cost and helper fee are always shown separately, never merged. This is how users learn the platform is fair.
- **Phone numbers stay hidden.** No design may expose or request a phone number. Communication happens in-app.
- **Thumb reach.** Used one-handed on mid-range Android phones. Primary actions live in the lower half.

06 Screen Inventory

ONBOARDING & ACCOUNT

6.1 Welcome

FIRST OPEN

One screen that says what Need It is in one line, then two paths: **Get things delivered** or **Earn on your way out**. This choice sets their starting role but never locks them in.

6.2 Register

ACCOUNT CREATION

College email, password, role choice. Includes a required acceptance of the platform policy and the prohibited-items list — present these readably, not as a wall of legalese. A checkbox plus a readable summary.

6.3 Verify Email

COLLEGE VERIFICATION

Tells the user a verification link was sent to their college email, and what to do next. Until verified, they can look but not post or accept — design the "waiting to be verified" state so it feels like a short step, not a punishment.

6.4 Login

RETURNING USERS

Email and password. Nothing else on this screen.

6.5 Profile

IDENTITY & PREFERENCES

Name, hostel block or preferred pickup point, verified email (shown as verified), role, contact preference. Editable except the verified email.

CUSTOMER

6.6 My Requests

CUSTOMER HOME

List of the user's requests with current status on each. A prominent create button. Empty state for a new user should encourage the first request — this is the app's activation moment.

6.7 Create Request

POSTING A NEED

Fields per item: **item name, quantity, expected price, notes, alternatives allowed (yes/no)**. Plus for the request: **needed-by time, urgent flag, pickup point**. Multiple items per request. If an item is prohibited, the user is told clearly and immediately, with the reason.

Fixed: the field list above, and immediate clear feedback on prohibited items. **Yours:** single-page form vs. step-by-step, item list interaction, how "urgent" looks.

6.8 Request Detail

TRACKING & ACTING ON A REQUEST

The busiest screen in the app. Shows: current status (prominently), items, the helper's first name once accepted, chat entry, and the actions valid right now — cancel (before purchase), approve a proposed price, approve or decline a replacement, pay, confirm handover. A status timeline works well here.

States to design: open & waiting · accepted & chatting · price proposed (needs your approval) · replacement proposed (needs your decision) · ready to pay · purchased · handover · completed · cancelled. Each looks different and offers different actions.

6.9 Pay

APPROVING AND PAYING

Reached after the customer approves the final price. Shows **item cost and helper fee as separate lines**, then the total, then the UPI payment action. After paying, a clear confirmation that money is held safely and the helper has been told to buy.

Fixed: item cost and fee shown as separate lines. Never merged.

HELPER

6.10 Feed

HELPER HOME — FINDING REQUESTS

Open requests only. Each card: item summary, category, **needed-by time (prominent)**, offered fee, pickup point. Requests that are locked, accepted, or done never appear here. This screen should feel alive — new requests appearing is the app's pulse.

6.11 My Orders

HELPER'S ACCEPTED WORK

Requests this helper has accepted, with status. The helper's counterpart to My Requests.

6.12 Order Detail (helper)

DOING THE JOB

Shows full request details — items with quantity, brand/size notes, price limit, suggested store if given, needed-by time, handover point — plus chat and the actions valid now: release the request, propose the actual price, mark an item unavailable (with a reason and a photo of a proposed replacement), mark purchased, show the handover code.

Once a helper accepts, the requester can no longer edit the request — so this screen can trust that what it shows is final.

6.13 Availability

"I'M GOING OUT"

Helper marks when they're heading out — date and time window — so requests can reach them at the right moment. Simple, secondary screen.

SHARED

6.14 Chat

CUSTOMER ↔ HELPER

Text and photos. Opens when a request is accepted; becomes read-only when the order completes. Photos matter: helpers send shelf photos for replacements. Never shows a phone number. Report access lives here too.

6.15 Handover

THE TRUST MOMENT

Both people, face to face at the pickup point, each enter a one-time code shown on the other's screen. When both confirm, the order completes and the helper's money is released.

Fixed copy requirement: the customer's confirm action must make clear that confirming releases the payment — e.g. "I've received my items — release payment." This wording is the product's core promise; design around it, don't soften it.

6.16 Report a Problem

SAFETY VALVE

Reachable from any active order and from chat. A short form — what happened, optional photo — that goes to the team. Confirm it was received and will be looked at by a human.

6.17 Notifications

KEEPING BOTH SIDES IN THE LOOP

In-app list of events: request accepted, price proposed, price change, item unavailable, ready for pickup, completed. Tapping one opens the relevant order at the relevant moment.

07 Feature Checklist by Screen

SCREEN	MUST HAVE	NICE TO HAVE (V1.1)
My Requests	Status per request, create button, empty state	Filter by status
Create Request	All fields, multi-item, prohibited feedback	Recent items, templates
Request Detail	Status timeline, chat entry, contextual actions	Helper ETA hint
Pay	Cost/fee split, total, UPI action, confirmation	—
Feed	Open requests, needed-by prominent, fee shown	Category filter, sort by time
Order Detail	Full request info, chat, contextual actions	Map hint for pickup point
Chat	Text, photos, read-only after completion	Quick replies ("on my way")
Handover	Code entry both sides, release copy	QR option
Notifications	Event list, deep link into order	Push toggles per type

08 What Is Fixed vs. What Is Yours

FIXED BY PRODUCT (NON-NEGOTIABLE)

- Status names and their order
- Actions available depend on current status — nothing more, nothing less
- Item cost and helper fee shown separately
- Handover confirm copy: confirming releases payment
- Prohibited items blocked with a clear reason
- No phone numbers anywhere
- Only verified students can post or accept

YOURS TO DESIGN

- Visual identity: color, type, iconography, motion
- Layout of every screen
- How status is visualized (chips, timeline, progress)
- Empty, loading, and error states
- Chat look and feel
- How the handover code is presented
- Navigation pattern (tabs, drawer, your call)
- Onboarding flow and tone

09 Practical Notes

- **Devices:** mid-range Android phones are the norm on campus; design and test for them first. iOS second but not optional.
- **Network:** campus Wi-Fi drops. Screens should handle loading and failure gracefully, and cached content is better than a spinner.
- **Photos:** chat and item-unavailable flows involve photos; assume they come from the camera in variable light.
- **One codebase, two platforms:** expect a cross-platform framework; avoid designs that depend on platform-specific widgets.
- **Dark mode:** students order at night. At minimum, don't design anything that breaks in dark mode.

10 Open Questions for You

- Tab bar or drawer for the customer/helper split — what do you recommend?
- Status visualization: chips, a vertical timeline, or a stepper? Show us a direction.
- Handover code: typed digits, or should we offer a QR scan option?
- Should "urgent" requests get a distinct visual treatment in the feed, or just a badge?

Bring answers as rough directions, not polished screens — we'll pick together, then you go deep.